



Amplify the future

Architettura POC

Proof of Concept · validazione rapida del valore

DOCUMENTO	VERSIONE	DATA	STATO
03 / 06	Rev. 1.0	20 Giugno 2026	POC / Hackathon

Milano

Via Franco Russoli, 6 · 20143

Perugia

Via f.lli Cairoli, 24 · 06125

Roma

Via di Castel Giubileo, 62 · 00138

Napoli

Via Giovanni Porzio, 4 · 80143

Bolzano

Via Giotto, 12 · 39100

Brindisi

Via Mulini, 13 · 72021 Francavilla
F. (BR)

Bologna

Via Aldo Moro, 16 · 40127

Tirana

Piazza Scanderbeg · 1001

Documento Confidenziale

Data revisione: 20 giugno 2026 **Versione:** POC Rev. 1.0

DISCLAIMER

Architettura finalizzata alla rapida validazione del valore e non alla messa in produzione senza ulteriore hardening tecnico, di sicurezza e operativo.

Questo documento descrive un'architettura pensata per dimostrare la fattibilità e il valore della soluzione Hello Work in un contesto di Hackathon. Le scelte tecniche privilegiano velocità di delivery e semplicità dimostrativa. Prima di qualsiasi passaggio in produzione è obbligatorio un Assessment Architetturale dedicato che copra sicurezza, resilienza, compliance e operatività.

1. Overview

HELLO WORK · ARCHITETTURA POC

Browser / Laptop — SPA

React 19 + TypeScript (Vite 6) · host Azure Static Web Apps

▼ HTTPS / REST

Azure Container Apps — ASP.NET Core 10 C#

API Gateway + Business Logic

/auth · MSAL

/profiles · 3 pilastri

/matches · tag-overlap

/workmatch · swipe FSM

/groups

/map

Azure SQL

Serverless · profili, match, gruppi

Azure AD / Entra

SSO · pre-popolamento profilo

Azure Blob

Avatar · asset statici

Azure Maps

Office map · clustering

Costo stimato POC < €15/mese (scale-to-zero + tier minimi).

2. Azure Services — POC

Servizio	SKU / Tier	Ruolo
Azure Container Apps	Consumption plan	Host ASP.NET Core 10 backend — scale to zero, zero infra management
Azure SQL Database	General Purpose 2 vCore (serverless)	Datastore principale: profili, match, swipe, gruppi
Azure Active Directory / Entra ID	Existing tenant	SSO aziendale, pre-population profilo da AAD claims
Azure Static Web Apps	Free tier	Host React 19 SPA (build Vite 6)
Azure Blob Storage	LRS Hot	Avatar upload, assets statici
Azure Maps	Gen 2 S0	Office Map — tile rendering + clustering pushpin API
Azure Container Registry	Basic	Registry immagini Docker per ASP.NET Core
Azure Key Vault	Standard	Secrets: DB connection string, Maps API key, MSAL client secret

Costo stimato POC: < €15/mese (scale-to-zero + tier minimi)

3. Stack Tecnico

Frontend — React 19 + TypeScript + Vite 6

```
src/
├── pages/
│   ├── Login.tsx      → MSAL redirect handler
│   ├── Home.tsx       → Discovery Feed
│   ├── Profile.tsx    → Profilo 3 pilastri + onboarding
│   ├── WorkMatch.tsx  → Swipe UI (Framer Motion cards)
│   ├── Groups.tsx     → Lista + suggerimenti gruppi
│   └── Map.tsx        → Azure Maps embed
├── components/
│   ├── SwipeCard.tsx  → WorkMatch card
│   ├── ProfilePillar.tsx → Pillar editor (Prof/Agentic/Human)
│   └── OfficeMap.tsx   → Azure Maps React wrapper
├── lib/
│   ├── msalConfig.ts  → MSAL.js configuration
│   ├── api.ts         → Typed fetch client (fetch + generics)
│   └── vite.config.ts  → Vite 6 configuration
```

Librerie chiave:

- `@azure/msal-browser` — SSO con Azure AD
- `@azure/msal-react` — React hooks per auth state
- `react-tinder-card` — Swipe gesture per WorkMatch

- `azure-maps-control` — Office Map
- `tailwindcss` + `shadcn/ui` — UI components rapidi

Backend — ASP.NET Core 10 C#

CSHARP

```
// Struttura moduli
HelloWork.Api/
├── Controllers/
│   ├── AuthController.cs    // Token introspection, AAD graph call
│   ├── ProfilesController.cs // CRUD profilo, 3 pilastri
│   ├── MatchesController.cs  // Tag-overlap scoring engine
│   ├── WorkMatchController.cs // Swipe state (liked/passed/matched)
│   ├── GroupsController.cs   // CRUD gruppi + suggestion
│   └── MapController.cs      // People cluster per ufficio
├── Models/                  // EF Core entities
├── DTOs/                    // Request/Response record types
├── Services/
│   ├── MatchingService.cs    // Jaccard similarity engine
│   └── AadGraphService.cs    // Microsoft Graph SDK
└── Infrastructure/
    ├── AppDbContext.cs      // EF Core + Azure SQL
    └── JwtValidationMiddleware.cs // MSAL JWT validation
```

Nota — Stack da validare (FastAPI)

Nel caso in cui, nell'evoluzione verso l'architettura enterprise, si valutasse l'adozione di **FastAPI (Python)** come runtime alternativo per il Matching Engine o altri servizi, tale scelta è **da validare in fase di design dettagliato** per garantire supportabilità nel contesto Microsoft-first.

Alternative raccomandate: Azure Functions (Flex Consumption) o **.NET Minimal API** — entrambi coerenti con l'ecosistema Azure e supportabili nel lungo periodo senza oneri aggiuntivi di runtime.

Database — Azure SQL Schema (core tables)

SQL

```
-- Profilo unificato
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT NEWID(),
  aad_oid TEXT UNIQUE NOT NULL,      -- Azure AD Object ID
  display_name TEXT,
  email TEXT,
  office_location TEXT,              -- Ufficio (Milano, Roma, Tirana...)
  avatar_url TEXT,
  -- Pillar 1: Professional (pre-populated da AAD)
  role TEXT,
  department TEXT,
  skills NVARCHAR(MAX),              -- tag array
  certifications NVARCHAR(MAX),
  -- Pillar 2: Agentic
  ai_tools NVARCHAR(MAX),            -- ['Claude','Copilot','n8n',...]
  ai_description TEXT,
  -- Pillar 3: Human
  hobbies NVARCHAR(MAX),
  interests NVARCHAR(MAX),
  created_at DATETIME2 DEFAULT GETUTCDATE(),
  updated_at DATETIME2 DEFAULT GETUTCDATE()
);
```

```
-- WorkMatch swipe state
CREATE TABLE workmatch_swipes (
  id UUID PRIMARY KEY DEFAULT NEWID(),
  swiper_id UUID REFERENCES users(id),
  target_id UUID REFERENCES users(id),
  direction TEXT CHECK (direction IN ('like','pass')),
  created_at DATETIME2 DEFAULT GETUTCDATE(),
  UNIQUE(swiper_id, target_id)
);

-- Mutual match result
CREATE TABLE matches (
  id UUID PRIMARY KEY DEFAULT NEWID(),
  user_a UUID REFERENCES users(id),
  user_b UUID REFERENCES users(id),
  match_score FLOAT,           -- tag overlap %
  status TEXT DEFAULT 'pending', -- pending/coffee_scheduled/connected
  created_at DATETIME2 DEFAULT GETUTCDATE()
);

-- Gruppi
CREATE TABLE groups (
  id UUID PRIMARY KEY DEFAULT NEWID(),
  name TEXT,
  description TEXT,
  tags NVARCHAR(MAX),          -- used for suggestion engine
  created_by UUID REFERENCES users(id),
  is_system_suggested BOOLEAN DEFAULT false,
  member_count INT DEFAULT 0
);

CREATE TABLE group_members (
  group_id UUID REFERENCES groups(id),
  user_id UUID REFERENCES users(id),
  joined_at DATETIME2 DEFAULT GETUTCDATE(),
  PRIMARY KEY (group_id, user_id)
);
```

4. Matching Engine (POC — Deterministic)

CSHARP

```
public class MatchingService
{
  /// <summary>
  /// Jaccard similarity across all 3 pillars with weights.
  /// No ML needed — deterministic, fast, explainable.
  /// </summary>
  public double ComputeMatchScore(User userA, User userB)
  {
    const double weightProfessional = 0.35;
    const double weightAgentic      = 0.40;
    const double weightHuman        = 0.25;

    static double Jaccard(IEnumerable<string> a, IEnumerable<string> b)
    {
      var setA = a.ToHashSet(StringComparer.OrdinalIgnoreCase);
      var setB = b.ToHashSet(StringComparer.OrdinalIgnoreCase);
      if (setA.Count == 0 && setB.Count == 0) return 0.0;
      int intersection = setA.Count(x => setB.Contains(x));
```

```

int union = setA.Union(setB).Count();
return (double)intersection / union;
}

double score =
    weightProfessional * Jaccard(userA.Skills, userB.Skills) +
    weightAgentic      * Jaccard(userA.AiTools, userB.AiTools) +
    weightHuman        * Jaccard(
        userA.Hobbies.Concat(userA.Interests),
        userB.Hobbies.Concat(userB.Interests));

return Math.Round(score, 4);
}
}

```

WorkMatch bilateral match trigger:

IF swipe(A→B) = 'like' AND swipe(B→A) = 'like'
 → INSERT INTO matches (user_a, user_b, score)
 → Send in-app notification: "Nuova connessione! Prenota un caffè virtuale "

5. Azure AD SSO Flow

AZURE AD · SSO FLOW (OAUTH 2.0 + OIDC)

Browser

React SPA

Azure AD

ASP.NET Core

- 1 L'utente clicca **Login** → la SPA avvia un **redirect MSAL** verso Azure AD.
- 2 Azure AD gestisce **consenso e autenticazione** e restituisce **id_token** + **access_token**.
- 3 La SPA chiama l'API con header **Bearer token**.
- 4 Il backend **valida il JWT** contro le chiavi AAD (JWKS).
- 5 Estrae **OID**, nome, email → **upsert** del profilo utente.
- 6 Profilo caricato e restituito alla SPA — zero friction al primo accesso.

Profilo pre-popolato da claim AAD: *displayName, mail, jobTitle, department, officeLocation*.

Profilo pre-popolato da AAD claims: **displayName**, **mail**, **jobTitle**, **department**, **officeLocation** — zero friction per l'utente al primo accesso.

6. Decisioni Tecniche Chiave (POC)

Decisione	Scelta	Razionale
Backend runtime	Azure Container Apps (Consumption)	Deploy in 5 min da Docker, scale-to-zero per demo
Backend framework	ASP.NET Core 10 C#	Performance nativa, typed endpoints, coerente con ecosistema Microsoft-first
Frontend framework	React 19 + TypeScript + Vite 6	SPA veloce, HMR istantaneo con Vite, ecosystem maturo
Auth flow	MSAL.js + Azure AD (existing tenant)	SSO aziendale nativo, no custom auth da costruire
Matching algorithm	Jaccard deterministic (C#)	Zero training data needed, esplicabile ai giudici
DB	Azure SQL Database + EF Core	Full-text search, JSON columns per tag arrays
Office Map	Azure Maps + static GeoJSON	Uffici hard-coded come GeoJSON → clustering nativo Maps
Swipe UI	react-tinder-card	30 min di integrazione, effetto wow garantito
CI/CD POC	GitHub Actions → ACR → Container Apps	Pipeline 3-step: <code>dotnet build</code> + <code>dotnet test</code> + Docker push

7. Compromessi del POC

Questa sezione documenta esplicitamente le semplificazioni accettate in fase POC. Ogni punto rappresenta un'area che **deve essere affrontata** prima della messa in produzione.

Area	Semplificazione accettata nel POC	Azione richiesta per la produzione
Resilienza	Single instance, no High Availability — un singolo restart abbatte il servizio	Zona Redundant deployment, health probes, autoscaling configurato
Sicurezza	TLS e AAD auth presenti, ma senza WAF, Private Endpoints, o rete privata per il data tier	Azure Front Door WAF, Private Endpoints per DB e Storage, network isolation
Monitoraggio	Application Insights di base (logs e tracing elementare), nessun alerting attivo	Alert rules su metriche critiche, Action Groups, SLO dashboard, on-call rotation
Scalabilità	Carico testato solo in contesto dimostrativo (< 50 utenti simultanei)	Load test documentato, autoscaling KEDA validato, match engine pre-computato
Operatività	Nessun runbook, nessuna procedura di incident management, nessun processo di backup verificato	Runbook operativi, RTO/RPO definiti, PITR testato, processo L1/L2 stabilito

8. Cosa non copre il POC

Le seguenti aree sono **fuori perimetro** per la fase POC e devono essere progettate e implementate prima di qualsiasi utilizzo in produzione:

- **Alta disponibilità e disaster recovery:** nessun meccanismo di failover automatico, nessuna strategia di recovery testata
- **Hardening completo di sicurezza:** assenza di WAF, Private Endpoints, Managed Identity su tutti i servizi, network segmentation
- **Compliance avanzata:** GDPR self-service (right-to-erasure automatizzato), audit log immutabile, DPA contrattuale con il fornitore cloud
- **Integrazioni enterprise estese:** sincronizzazione HRIS, calendar integration via Graph API per coffee chat scheduling, Teams Tab / Viva Connections
- **Supporto a carichi non ancora validati:** throughput, concorrenza e dimensionamento del DB non sono stati oggetto di load test
- **Procedure operative e supporto:** nessun processo L1/L2, nessuna documentazione di supporto, nessuna gestione degli incidenti formalizzata

Hello Work — POC Architecture AGIC — Giugno 2026